or remove elements anywhere from the *Stack!* Here is a code example that illustrates this problem:

```
Vector<String> vectorStack = new Stack<String>();
vectorStack.addElement("one");
vectorStack.addElement("two");
vectorStack.addElement("three");
vectorStack.removeElementAt(1);
System.out.println(vectorStack.size());
// prints: 2
```

As you can see, in this program, we can remove the element from the middle of the *Stack* (with the call *removeElementAt(1)*), and treat *Stack* as if it were a *Vector*! So, how do we treat a *Stack* as a stack when we have a *Vector* reference? One way is for the user to check the dynamic type (i.e., what class type the object reference points to at runtime) of the object—and if that is a *Stack*, do a downcast to *Stack* and apply operations such as *push* and *pop*. This is too much of a workaround because of a design mistake in the Java library. In other words, *Stack* could have been declared as an interface, and different *Stack* implementations could have been derived from it. Or else, *Stack* could have been implemented using *Vector* as the data container, i.e., using containment instead of inheritance.

Another example of LSP violation is in the case of the *Property* class, which extends *Hashtable*. A *Hashtable* can take any non-null values as key and values. A *Property* object can take only a *String* as key or value. The *Property* class has methods like *getProperty* and *setProperty* to get and set property values. However, we still have access to methods such as *put* and *putAll* from *Hashtable*, which we can use to put keys/values of any type. When we attempt to put non-*String* keys or values in a *Property*, it appears to work fine:

```
Hashtable extnNos = new Properties();
extnNos.put("Kathy", "3542");
extnNos.put("Joel", "4433");
// mistake in the following statement:
//            3224 typed instead of "3224"
extnNos.put("Joshua", 3224);
System.out.println(extnNos);
```

This code segment prints:

```
{Kathy=3542, Joshua=3224, Joel=4433}
```

However, Java documentation calls such *Property* objects "compromised". Method calls such as *store*, *save* or *list* fail when called on "compromised" *Property* objects. Here is a slightly modified version of the same code using Properties. list method instead of System.out.println:

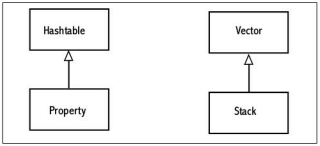

Figure 1: Two LSP violation examples from JDK

```
Hashtable extnNos = new Properties();
extnNos.put("Kathy", "3542");
extnNos.put("Joel", "4433");
// mistake in the following statement:
//            3224 typed instead of "3224"
extnNos.put("Joshua", 3224);
((Properties)extnNos).list(System.out);
```

This code results in a crash:

```
-- listing properties --
Kathy=3542
Exception in thread "main" java.lang.ClassCastException:
                java.lang.Integer cannot be cast to java.lang.
String
        at java.util.Properties.list(Unknown Source)
        at UseProperties.main(UseProperties.java:9)
```

This design—*Property* extending *Hashtable*—violates LSP. In this case, a *Property* would have been better implemented using a *Hashtable*, i.e., without sharing the inheritance relationship with each other. Because of this design mistake, users of the *Property* class need to take more care in using the class correctly.

Since JDK is a library—a published API—it is not possible to correct these design mistakes. As users, we cannot do anything about these design mistakes in JDK, but we can learn from them; following established design principles or rules can help create better designs, and not following them can be costly. END

By: S G Ganesh

The author works for Siemens (Corporate Research & Technologies, Bangalore). You can reach him at *sgganesh@gmail.com*.